READ ME **Assumptions, and the one structural bet**

The bet: your docs are already structured like a game — requirements, entities, API, high-level design, deep dives, then an explicit *Mid / Senior / Staff+* rubric at the end. So the levels don't need inventing. Each deep dive is a level; the rubric is the difficulty tier; the "Key Takeaways" are the win conditions. Gamification here is *extraction*, not decoration.

Static is fine. Everything below runs client-side. Progress is a single JSON blob in `localStorage` keyed by doc slug. Challenge definitions ship as a small JSON file per doc, generated at build time from the headings and takeaways you already write. See `1h` for the three storage tiers and when each one starts to hurt.

Intensity ~30/100. No confetti, no sound, no cartoon mascot. The reward is a filling bar and an honest score against the Staff+ rubric. Everything below is interactive — click it.

OPEN QUESTIONS FOR YOU

1. Does the sidecar ship on all 122 docs as an empty shell, or only appear on the 5 flagship docs?
2. Are the SVG diagrams authored by you (so I can add hotspots), or exported?
3. Is a "wrong" answer allowed to be logged for spaced repetition, or is that too much bookkeeping?

1 Eight atomic pieces — one shell, six mechanics, one progress model

1a The sidecar — persistent HUD beside the article, tier-scoped

PROBLEM BREAKDOWNS

Bitly

6,549 words · 2 diagrams · 8 figures

⚡ 2) How can we ensure that redirects are fast?

When dealing with a large database of shortened URLs, finding the right match quickly becomes crucial. Without optimization our system would check every pair of short and original URLs — a full table scan — which is incredibly slow as the number of URLs grows into the billions.

To avoid it we use indexing. Think of an index like a book's table of contents: a separate, sorted list of short codes, each with a pointer to where the full row lives. A B-tree index gives O(log n) lookup, and making the short code the primary key gets us uniqueness for free.

> Memory access is ~1,000× faster than SSD and ~100,000× faster than HDD.

Even indexed, a single instance struggles: 100M DAU × 5 redirects/day = 5,787 rps averaged, and peaks push that toward 600k. So we put an in-memory cache between the application server and the database…

PRACTICE

Senior track
2 of 5 checkpoints

Mid | Senior | Staff+

- Requirements triage +25
  8 items · 90 seconds
  § Understanding the Problem
- Tradeoff duel +30
  Hash + base62 vs atomic counter
  § Deep dive 1
- Capacity ladder +20
  5 redirects/user → 600k rps
  § Deep dive 2
- Architecture builder +40
  Split read and write services
  § Deep dive 3

Timed mode
Due for review 3 cards
Vault XP 1,240 · L7

1b Tradeoff duel — pick, then defend

TRADEOFF DUEL Senior · Deep dive 1

1B URLs, and short codes must never collide. Which generator?

Hash + base62	Counter + base62
SHA-256, take first 8	Redis INCR, encode

↺ reset

1c Requirements triage — above or below the line, on a clock

REQUIREMENTS TRIAGE untimed

Bitly — scope it in 90 seconds.
Core requirement, or below the line?

Submit a long URL, get a short one	core	below
Redirect a short code to the original	core	below
Optional custom alias	core	below
Optional expiration date	core	below
Click analytics and geo data	core	below
User accounts and auth	core	below
Spam / malicious URL filtering	core	below
Real-time analytics consistency	core	below

↺ reset

1d Capacity ladder — three rungs of back-of-envelope

CAPACITY LADDER

100M DAU. 5 redirects each. What does the read path have to survive?

1 100M DAU × 5 redirects — reads per day?
50M | 500M | 5B

2 Averaged over 86,400 seconds — reads per second?
580 | 5.8K | 58K

3 Peaks run ~100× the average. Design target?
60K | 600K | 6M

↺ reset

1e Architecture builder — tap a component, tap a slot, get graded

ARCHITECTURE BUILDER Senior · the read path

Build the redirect path so a hot short code never touches disk.

Browser → empty → empty → empty → Postgres

PALETTE
CDN edge | Load balancer | Read service | Write service | Redis cache | Kafka | Redis counter | Elasticsearch

Check design ↺ reset

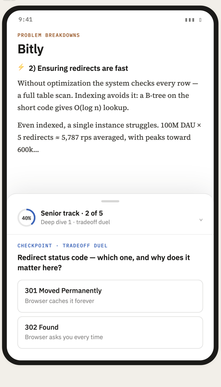
1f Spot the bottleneck — a dashboard, one wrong number

SPOT THE BOTTLENECK Staff+ · incident replay

p99 redirect latency just went from 40ms to 900ms.
Tap the node you'd fix first.

CDN edge OK	Read service ×12 OK
hit ratio 88% p99 12ms	cpu 34% p99 880ms
Redis cache WARN	**Postgres primary CRIT**
hit ratio 41% ↓ evictions 9.2k/s	cpu 96% 3.4k qps ↑

1g Mobile — the sidecar becomes a bottom sheet



1h Progress on a static site — the vault map, and where localStorage runs out

THE VAULT, AS A SKILL MAP

Caching 82%	Redis 74%	CAP 90%	Sharding 61%
Indexing 44%	Cons. hashing 39%	Kafka 12%	Networking 22%

11-day streak 14d

THREE WAYS TO KEEP IT, IN ORDER OF PAIN

Device-local SKIP THIS
One JSON blob in localStorage, keyed by doc slug: attempts, last score, next review date. Zero infrastructure, survives a rebuild, dies when they switch to their laptop.

Portable code ONE AFTERNOON
Same blob, base64'd into an "export progress" string they paste on the other device — or into a #hash they bookmark. Still zero backend, and it makes the streak feel worth protecting.

Passphrase sync WHEN THEY ASK
One serverless route and a KV store, keyed by a passphrase instead of an account. No signup, no email, no password reset. Only worth it once a leaderboard exists.

Try next: "build `1a` out as a real drop-in widget" · "more mechanics like `1e`, drag instead of tap" · "show me the boss round — a timed Staff+ mock" · "what does the JSON per doc look like?"